# Class Templates and Binomial Lattices

FRE6883 – Recitation 9

One lattice design, many stored types

# From a data structure to a reusable type pattern

`vector<vector<double>>`

price tree

`vector<vector<bool>>`

stopping tree

↓

`BinLattice<Type>`

**By the end**

You can read and instantiate function and class templates, complete `BinLattice<Type>`, and pass typed lattices to the European and American pricing algorithms.

Templates remove duplicated structure while preserving compile-time type checking.

# The duplication we want to remove

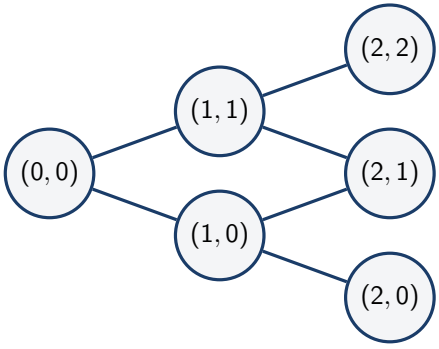
```
class PriceLattice {
    std::vector<
        std::vector<double> > data;
    // SetNode(..., double x)
};
```

```
class StoppingLattice {
    std::vector<
        std::vector<bool> > data;
    // SetNode(..., bool x)
};
```

**Only the element type changes**

The triangular shape, indexing, allocation, and display loops are otherwise the same.

# What the lattice represents




row $n$ has $n + 1$ nodes;
a node is addressed by $(n, i)$.

The same coordinates may store an asset price, option value, stopping flag, hedge ratio, or bond position.

# 01 | Spot the duplicated function – Problem

**GUIDED CODE** Three approaches

What varies?

```
void PrintInt(int value)
{ std::cout << value << std::endl; }

void PrintDouble(double value)
{ std::cout << value << std::endl; }

void PrintString(const std::string& value)
{ std::cout << value << std::endl; }
```

**Question**

Which part of the algorithm is identical? Which part should become a template parameter?

# 01 | Spot the duplicated function – Solution

```
template <typename Type>
void Print(const Type& value)
{
    std::cout << value << std::endl;
}
```

**Template parameter**

Type is a placeholder. The compiler substitutes a concrete type when the function is used.

This one definition can generate functions for `int`, `double`, `std::string`, and other printable types.

# Naive, overload, or template?

| Approach | Definitions | Best fit |
|---|---|---|
| different names | one per type | behavior differs by type |
| overloading | same name, one body per signature | a few type-specific bodies |
| template | one parameterized definition | same algorithm for many types |

**Theme 7 choice**

The lattice algorithm is identical for `double` and `bool`, so a class template expresses the shared design directly.

# 02 | Predict template deduction – Problem

**GUIDED CODE** Function template — Identify Type at each call

```
template <typename Type>
Type Square(Type x)
{
    return x * x;
}

std::cout << Square(4) << std::endl;       // A
std::cout << Square(2.5) << std::endl;     // B
std::cout << Square<double>(3) << std::endl; // C
```

**Questions**

What type is instantiated for A, B, and C? What values are printed?

# 02 | Predict template deduction – Solution

| Call | Instantiated Type | Result |
|---|---|---|
| `Square(4)` | `int` | `16` |
| `Square(2.5)` | `double` | `6.25` |
| `Square<double>(3)` | `double` | `9.0` |

**Deduction versus explicit argument**

The compiler normally deduces the function template argument from the call. The third call supplies `double` explicitly.

# 03 | Find the template type error – Problem

**GUIDED CODE** One parameter, one deduced type — Will this compile?

```
template <typename Type>
Type Larger(Type a, Type b)
{
    return (a > b) ? a : b;
}

std::cout << Larger(3, 4.5) << std::endl;
```

**Question**

The first argument suggests `int`; the second suggests `double`. How can the call be made unambiguous?

# 03 | Find the template type error – Solution

```
// Use matching argument types:
std::cout << Larger(3.0, 4.5) << std::endl;

// Or choose the template argument explicitly:
std::cout << Larger<double>(3, 4.5) << std::endl;
```

**Compile-time contract**

Both parameters of `Larger<Type>` must use the same substituted `Type`. The compiler does not guess which conflicting deduction should win.

# A class template parameterizes an entire type

```
template <typename Type>
class Box {
private:
    Type value;
public:
    Box(Type value_) : value(value_) {}
    Type Get() const { return value; }
};
```

**Generated class types**

`Box<int>` and `Box<double>` are distinct class types produced from the same template definition.

# 04 | Instantiate the class template – Problem

**GUIDED CODE** Explicit class arguments — Complete both declarations

```
/* an integer Box containing 7 */
/* a double Box containing 2.5 */

std::cout << integerBox.Get() << std::endl;
std::cout << doubleBox.Get() << std::endl;
```

**Question**

Unlike the function-template calls above, where must the class template argument appear?

# 04 | Instantiate the class template – Solution

```
Box<int> integerBox(7);
Box<double> doubleBox(2.5);

std::cout << integerBox.Get() << std::endl;
std::cout << doubleBox.Get() << std::endl;
```

**Class template argument**

The course syntax writes the concrete element type inside angle brackets: `Box<int>` or `BinLattice<double>`.

# 05 | Are these assignments valid? – Problem

**GUIDED CODE** Distinct generated types — Vote for each line

```
Box<int> a(7);
Box<int> b(9);
Box<double> c(2.5);

b = a;  // A
c = a;  // B
```

**Question**

Does sharing one template definition make `Box<int>` and `Box<double>` assignment-compatible types?

# 05 | Are these assignments valid? – Solution

```
b = a;  // valid: Box<int> to Box<int>
c = a;  // error: Box<int> and Box<double> are distinct
```

**Instantiation model**

The compiler produces a separate class type for each template argument. Shared source code does not erase the element type.

# 06 | Complete an out-of-class member definition – Problem

**GUIDED CODE** Template syntax

Fill both missing pieces

```
template <typename Type>
class Box {
public:
    Type Get() const;
private:
    Type value;
};

/* template declaration */
Type /* class scope */ Get() const
{
    return value;
}
```

# 06 | Complete an out-of-class member definition – Solution

```
template <typename Type>
Type Box<Type>::Get() const
{
    return value;
}
```

**Read the qualified name**

`Box<Type>::Get` is the `Get` member of the class-template specialization currently being defined.

Template definitions are normally kept in header files so the compiler can see them when it instantiates a concrete type.

# A template may have type and value parameters

```
template <typename Type, int Size>
class FixedStack {
private:
    Type data[Size];
    int top;
public:
    FixedStack() : top(0) {}
};

FixedStack<double, 100> prices;
FixedStack<int, 20> indices;
```

**Two compile-time arguments**

Type selects the element type; Size selects the array length. Topic 7's BinLattice needs only the type parameter.

# The first lattice stores only `double`

```
class BinLattice {
private:
    int N;
    std::vector< std::vector<double> > Lattice;
public:
    BinLattice() : N(0) {}
    BinLattice(int N_);
    void SetN(int N_);
    void SetNode(int n, int i, double x);
    double GetNode(int n, int i) const;
    void Display() const;
};
```

# 07 | Identify state and interface – Problem

**GUIDED CODE** BinLattice design

Answer in four passes

```
int N;
std::vector< std::vector<double> > Lattice;

void SetN(int N_);
void SetNode(int n, int i, double x);
double GetNode(int n, int i) const;
void Display() const;
```

**Questions**

What does each data member represent? Which function allocates the triangle? Which function writes a node? Which function reads one?

# 07 | Identify state and interface – Solution

### State

`N` stores the final step; `Lattice` stores rows 0 through $N$.

### Shape

`SetN` resizes row $n$ to $n + 1$ nodes.

### Mutation

`SetNode(n,i,x)` writes one value.

### Observation

`GetNode(n,i)` and `Display()` read stored values.

**Template target**

Parameterize the stored node type while keeping these coordinates and operations unchanged.

# 08 | Turn `BinLattice` into a template – Problem

**GUIDED CODE** Class template — Replace every type-specific occurrence

```
class BinLattice {
private:
    int N;
    std::vector< std::vector<double> > Lattice;
public:
    void SetNode(int n, int i, double x);
    double GetNode(int n, int i) const;
};
```

**Task**

Add the template declaration and replace the three occurrences of `double` that describe stored node values.

# 08 | Turn `BinLattice` into a template – Solution

```
template <typename Type>
class BinLattice {
private:
    int N;
    std::vector< std::vector<Type> > Lattice;
public:
    void SetNode(int n, int i, Type x);
    Type GetNode(int n, int i) const;
};
```

**Only node values are generic**

The time and node indices remain integers. The template parameter controls the value stored at each $(n, i)$.

**GUIDED CODE** Constructor and SetN

Fill the row sizes

```
BinLattice(int N_) : N(N_)
{
    Lattice.resize(/* rows */);
    for (int n = 0; n <= N; ++n)
        Lattice[n].resize(/* nodes in row n */);
}

void SetN(int N_)
{
    N = N_;
    /* allocate the same triangular shape */
}
```

# 09 | Complete triangular allocation – Solution

```
BinLattice(int N_) : N(N_)
{
    Lattice.resize(N + 1);
    for (int n = 0; n <= N; ++n)
        Lattice[n].resize(n + 1);
}

void SetN(int N_)
{
    N = N_;
    Lattice.resize(N + 1);
    for (int n = 0; n <= N; ++n)
        Lattice[n].resize(n + 1);
}
```

# 10 | Complete node access – Problem

**GUIDED CODE** Generic getter and setter

Use Type correctly

```
void SetNode(int n, int i, /* node type */ x)
{
    /* write x at (n,i) */
}

/* node type */ GetNode(int n, int i) const
{
    /* return the value at (n,i) */
}
```

# 10 | Complete node access – Solution

```
void SetNode(int n, int i, Type x)
{
    Lattice[n][i] = x;
}

Type GetNode(int n, int i) const
{
    return Lattice[n][i];
}
```

**Compile-time checking**

`BinLattice<double>::SetNode` expects a `double`; `BinLattice<bool>::SetNode` expects a Boolean-convertible value.

# 11 | Complete `Display` – Problem

**GUIDED CODE** Nested traversal

Fill both loop bounds

```
void Display() const
{
    for (int n = 0; /* all rows */; ++n)
    {
        for (int i = 0; /* all nodes in row n */; ++i)
            std::cout << GetNode(n, i) << " ";
        std::cout << std::endl;
    }
}
```

# 11 | Complete `Display` – Solution

```
void Display() const
{
    for (int n = 0; n <= N; ++n)
    {
        for (int i = 0; i <= n; ++i)
            std::cout << GetNode(n, i) << " ";
        std::cout << std::endl;
    }
}
```

The same traversal works for every printable `Type`; only the stored values differ.

**GUIDED CODE** Class template arguments

Declare three objects

```
/* price tree storing double values */
/* stopping tree storing bool values */
/* integer lattice with N = 4 */

priceTree.SetN(N);
stoppingTree.SetN(N);
integerTree.SetNode(2, 1, 7);
```

# 12 | Instantiate typed lattices – Solution

```
BinLattice<double> priceTree;
BinLattice<bool> stoppingTree;
BinLattice<int> integerTree(4);

priceTree.SetN(N);
stoppingTree.SetN(N);
integerTree.SetNode(2, 1, 7);
```

**Distinct purposes, shared implementation**

One class-template definition supplies the triangular storage and interface for all three node types.

# The pricing interface states each lattice type

```
double PriceBySnell(
    const BinomialTreeModel& Model,
    BinLattice<double>& PriceTree,
    BinLattice<bool>& StoppingTree);
```

**Price tree**

Each node stores the American option value as a `double`.

**Stopping tree**

Each node stores an exercise decision as a `bool`.

# 13 | Initialize both terminal rows – Problem

**GUIDED CODE** PriceBySnell

Complete the typed setter calls

```
PriceTree.SetN(N);
StoppingTree.SetN(N);

for (int i = 0; i <= N; ++i)
{
    double payoff = pOption->Payoff(
        Model.CalculateAssetPrice(N, i));

    /* store payoff at (N,i) */
    /* store the terminal stopping convention */
}
```

# 13 | Initialize both terminal rows – Solution

```
PriceTree.SetN(N);
StoppingTree.SetN(N);

for (int i = 0; i <= N; ++i)
{
    double payoff = pOption->Payoff(
        Model.CalculateAssetPrice(N, i));

    PriceTree.SetNode(N, i, payoff);
    StoppingTree.SetNode(N, i, true);
}
```

The lecture code marks every expiry node as stopping. When `bool` instead means "positive exercise payoff," store `payoff > 0.0`; document the convention.

**GUIDED CODE** Backward induction — Complete the getters

```
double contVal =
    (q * /* up child from PriceTree */
     + (1.0 - q) * /* down child from PriceTree */)
    / Model.GetR();

double exerciseVal = pOption->Payoff(
    Model.CalculateAssetPrice(n, i));
```

**Question**

Which lattice coordinates contain the up and down values after row $n+1$ has been computed?

# 14 | Read the two child values – Solution

```
double contVal =
    (q * PriceTree.GetNode(n + 1, i + 1)
     + (1.0 - q) * PriceTree.GetNode(n + 1, i))
    / Model.GetR();

double exerciseVal = pOption->Payoff(
    Model.CalculateAssetPrice(n, i));
```

**Encapsulated storage**

The pricing algorithm uses lattice coordinates through `GetNode`; it no longer depends directly on a vector-of-vector representation.

# 15 | Store value and policy – Problem

**GUIDED CODE** Snell decision

Complete the four setter calls

```
/* begin with immediate exercise */
PriceTree.SetNode(n, i, /* ? */);
StoppingTree.SetNode(n, i, /* positive exercise? */);

if (contVal > PriceTree.GetNode(n, i))
{
    /* replace price by continuation */
    /* record no exercise */
}
```

# 15 | Store value and policy – Solution

```
PriceTree.SetNode(n, i, exerciseVal);
StoppingTree.SetNode(n, i, exerciseVal > 0.0);

if (contVal > PriceTree.GetNode(n, i))
{
    PriceTree.SetNode(n, i, contVal);
    StoppingTree.SetNode(n, i, false);
}
```

**One coordinate, two typed facts**

`PriceTree(n,i)` stores the optimal value; `StoppingTree(n,i)` stores which action produced it.

# 16 | Wire the driver – Problem

**GUIDED CODE** Typed objects — Complete three declarations and one call

```
Put put(N, K);

/* double lattice for option values */
/* bool lattice for exercise decisions */
/* calculator selecting put */

double value = /* call PriceBySnell */;
```

**Question**

Which objects are modified by the pricing function, and why are they passed by non-const reference?

# 16 | Wire the driver – Solution

```
Put put(N, K);

BinLattice<double> priceTree;
BinLattice<bool> stoppingTree;
OptionCalculation calc(&put);

double value = calc.PriceBySnell(
    binModel, priceTree, stoppingTree);

std::cout << value << std::endl; // about 11.724
priceTree.Display();
stoppingTree.Display();
```

**GUIDED CODE** Overloaded PriceByCRR

Replace the flat vector operations

```
double PriceByCRR(
    const BinomialTreeModel& Model,
    BinLattice<double>& PriceTree)
{
    int N = pOption->GetN();
    /* allocate PriceTree */
    /* initialize row N */
    /* backward-induct through every row */
    return /* root node */;
}
```

**Task**

Name the lattice operations required in each of the four blanks.

# 17 | Store a full European price tree – Solution

```
PriceTree.SetN(N);
for (int i = 0; i <= N; ++i)
    PriceTree.SetNode(N, i, pOption->Payoff(
        Model.CalculateAssetPrice(N, i)));

for (int n = N - 1; n >= 0; --n)
    for (int i = 0; i <= n; ++i)
        PriceTree.SetNode(n, i,
            (q * PriceTree.GetNode(n + 1, i + 1)
             + (1.0 - q) * PriceTree.GetNode(n + 1, i))
            / Model.GetR());

return PriceTree.GetNode(0, 0);
```

# The template changes storage reuse, not complexity

| Computation | Time | Storage |
|---|---|---|
| root price with one vector | $O(N^2)$ | $O(N)$ |
| full `BinLattice<double>` | $O(N^2)$ | $O(N^2)$ |
| price and stopping lattices | $O(N^2)$ | $O(N^2)$ |

**What the template buys**

One tested lattice implementation supports several node types. It does not reduce the number of binomial nodes that must be evaluated.

# 18 | Diagnose four template mistakes – Problem

**GUIDED CODE** Final debugging check

One minute, four fixes

```
BinLattice priceTree;                  // A
BinLattice<double, 8> fixedTree;       // B
BinLattice<bool> stoppingTree;

priceTree.SetNode(2, 1, true);         // C
double x = stoppingTree.GetNode(2, 1);// D
```

**Assumption**

`BinLattice` has exactly one template parameter: `Type`. Identify the issue or misleading conversion on each line.

# 18 | Diagnose four template mistakes – Solution

```
BinLattice<double> priceTree;          // supply Type
BinLattice<double> fixedTree(8);       // N is constructor input
BinLattice<bool> stoppingTree;

priceTree.SetNode(2, 1, 1.0);          // store a price
bool x = stoppingTree.GetNode(2, 1);   // read a flag
```

**Exit ticket**

State which parts of `BinLattice<Type>` depend on `Type`, which remain integer indices, and why the compiler creates distinct lattice types.

# Reserve workshop

Build and analyze European and American straddle lattices.

# R1 | Implement the straddle payoff – Challenge

**GUIDED CODE** Previous-year workshop

Complete the polymorphic payoff

```
class Straddle : public Option {
private:
    double K;
public:
    Straddle(int N_, double K_) : Option(N_), K(K_) {}
    double Payoff(double z) const;
};

double Straddle::Payoff(double z) const
{
    /* return |z - K| */
}
```

# R1 | Implement the straddle payoff – Solution

```cpp
#include <cmath>

double Straddle::Payoff(double z) const
{
    return std::fabs(z - K);
}
```

**Equivalent decomposition**

$$|z - K| = \max(z - K, 0) + \max(K - z, 0).$$

A straddle combines a call and a put payoff at the same strike.

# R2 | Create and price three typed lattices – Challenge

**GUIDED CODE** Previous-year workshop

Complete the driver

```
Straddle straddle(N, K);

/* European price lattice */
/* American price lattice */
/* American stopping lattice */

OptionCalculation calc(&straddle);
double euro = /* overloaded PriceByCRR */;
double amer = /* PriceBySnell */;

std::cout << "Premium = " << /* ? */ << std::endl;
```

# R2 | Create and price three typed lattices – Solution

```
BinLattice<double> europeanTree;
BinLattice<double> americanTree;
BinLattice<bool> stoppingTree;

OptionCalculation calc(&straddle);
double euro = calc.PriceByCRR(binModel, europeanTree);
double amer = calc.PriceBySnell(
    binModel, americanTree, stoppingTree);

std::cout << "European = " << euro << std::endl;
std::cout << "American = " << amer << std::endl;
std::cout << "Premium = " << amer - euro << std::endl;
```

Expected values: European 33.107, American 33.348, premium 0.241.

# R3 | Count exercise nodes – Challenge

**GUIDED CODE** Previous-year workshop

Read the Boolean lattice

```
for (int n = 0; n < N; ++n)
{
    int count = 0;
    for (int i = 0; i <= n; ++i)
    {
        /* inspect stoppingTree at (n,i) */
    }
    std::cout << "step " << n << ": " << count;
}
```

# R3 | Count exercise nodes – Solution

```
for (int n = 0; n < N; ++n)
{
    int count = 0;
    for (int i = 0; i <= n; ++i)
    {
        if (stoppingTree.GetNode(n, i))
            ++count;
    }
    std::cout << "step " << n << ": " << count;
}
```

**Expected counts**

For steps 0 through 7: $0, 0, 0, 0, 1, 1, 2, 3$.

# R4 | Stream a lattice – Challenge

**GUIDED CODE** Course extension

Complete the friend operator

```
friend std::ostream& operator<<(
    std::ostream& out, const BinLattice& lattice)
{
    for (int n = 0; n <= lattice.N; ++n)
    {
        for (int i = 0; i <= n; ++i)
            /* write lattice.Lattice[n][i] to out */;
        out << std::endl;
    }
    return /* ? */;
}
```

# R4 | Stream a lattice – Solution

```
friend std::ostream& operator<<(
    std::ostream& out, const BinLattice& lattice)
{
    for (int n = 0; n <= lattice.N; ++n)
    {
        for (int i = 0; i <= n; ++i)
            out << std::setw(15) << lattice.Lattice[n][i];
        out << std::endl;
    }
    return out;
}
```

Returning the stream permits chained output such as `std::cout << priceTree << std::endl;`.

# R5 | One-period replication – Optional finance challenge

Suppose a stock is worth 100 today and will be worth 137 or 83 next period. A bond grows from 100 to 102. A call with strike 105 pays 32 or 0.

**Solve for stock and bond positions**

Find $x$ shares and a time-zero bond position $y$ such that

$$137x + 1.02y = 32, \qquad 83x + 1.02y = 0.$$

**Solution**

$x = 32/(137 - 83) \approx 0.5926$ and $y \approx -48.22$. The option is replicated by stock financed partly through borrowing.

# Recitation 9 cheat sheet

| Concept | Working rule |
|---|---|
| Function template | compiler deduces or receives a type argument per call |
| Class template | angle brackets select a concrete generated class type |
| `template<typename T>` | declare the placeholder used in the definition |
| `Class<T>::member` | qualify an out-of-class template member definition |
| `BinLattice<double>` | lattice of numeric node values |
| `BinLattice<bool>` | lattice of stopping decisions |
| Row `n` | contains nodes 0 through `n` |
| Output lattice reference | lets the pricer fill a caller-owned tree |